

- メソッドのシグネチャが一致しない場合のミスを防ぐ.
- Java では明示的なメソッドシグネチャが必要だが,Dart はより柔軟.

```
class Parent {
    void build() {
        System.out.println("Parent build method");
    }
}

class Child extends Parent {
    @Override
    void build() { // Overriding the parent method
        System.out.println("Child build method");
    }
}
```

- 説明:
 - ◇ @Override → Child の build() が Parent の build() を正しくオーバーライドしていることを保証する.
 - ◇ メソッド名が間違っていた場合,Java はエラーを出す.
- 主な違い:

機能	Java	Flutter
@override の用途	OOP メソッドのオーバーライド	Flutter ウィジェットのオーバーライド
エラー防止	誤ったメソッドシグネチャを防ぐ	タイプミスの検出を助ける
柔軟性	厳格なメソッド定義	より柔軟だが,@override で明確

ステップ6：build() メソッドとBuildContext の探索

a. build()メソッドとは？

- build() メソッドは,ウィジェットが画面にどのように表示されるかを定義する.
- “ウィジェットツリー” (UI を構成するウィジェットの階層構造) を返す.
- UI が更新される必要があるときに,build() メソッドが再び呼び出される.

```
class MyApp extends StatelessWidget {
    @override
    Widget build(BuildContext context) {
        return MaterialApp(
            home: Scaffold(
                body: Center(
                    child: Text('Hello, Flutter!'),
                ),
            ),
        );
    }
}
```

```
}
```

- 説明:
 - ◇ build() は MaterialApp ウィジェットを返し,アプリの UI 全体を定義する.
 - ◇ MaterialApp の中に Scaffold,Center,Text などのウィジェットが含まれる.
 - ◇ この構造がウィジェットツリーを形成し,UI のレイアウトを決める.

b. BuildContext とは？

- BuildContext は,ウィジェットがウィジェットツリーのどこにあるかを示す情報を提供する.
- 継承されたウィジェットにアクセスしたり,ナビゲーションを管理したりできる.
- build() メソッドの必須パラメータである.

c. build()が UI を更新し続ける仕組み

- Flutter は UI の変化を検出すると build() を再び呼び出す.
- UI の最新状態を確実に表示するために build() を利用する.
- Flutter は宣言的 UI モデルを採用しており,手動で UI を変更するのではなく,再構築する.

d. 比較：Java における UI の構築方法との違い

- Java (Android) では,通常,UI の更新は View オブジェクトの変更によって行われる.
- Flutter では,UI は宣言的に構築され,build() を使って再構築される.
- Flutter のアプローチにより,UI の更新が構造化され,予測しやすくなる.

```
@Override
protected View build(Context context) {
    TextView textView = new TextView(context);
    textView.setText("Hello, Android!");
    return textView;
}
```

- 主な違い

機能	Java (Android)	Flutter (Dart)
UI の構造	XML や Java コードで View を作成	build() メソッドでウィジェットツリーを返す
状態管理	View を手動で更新	状態が変わると UI が自動更新
柔軟性	View 階層を変更する必要がある	build() によりシンプルに管理

ステップ 7：Flutter における return の役割

a. build()における return の役割とは？

- build() メソッド内で return を使って,ウィジェットが表示する内容を指定する.